

Problema 02: Análise de Estabilidade e Custo em Sistemas de Abastecimento de Água

Diogo Estéfano Cambraia Freitas | Engenharia de Telecomunicações

LLM 1 | Chat GPT

```
# =====
# ANÁLISE DE ESTABILIDADE E CUSTO
# EM SISTEMAS DE ABASTECIMENTO DE ÁGUA
# =====

import pandas as pd
import matplotlib.pyplot as plt

# =====
# ENTRADA DE DADOS
# =====

print("=" * 60)
print("ANÁLISE DE ESTABILIDADE E CUSTO")
print("SISTEMAS DE ABASTECIMENTO DE ÁGUA")
print("=" * 60)

N = int(input("\nInforme o número de bairros: "))

# Listas para armazenamento
bairros = []
demandas = []
capacidades = []
volumes_iniciais = []
perdas_percentuais = []
custos_bombeamento = []

# =====
# COLETA DAS INFORMAÇÕES
# =====

for i in range(N):
    print(f"\n--- Bairro {i+1} ---")

    nome = input("Nome do bairro: ")
```

```

demanda = float(input("Demanda diária estimada (m³): "))

capacidade = float(input("Capacidade do reservatório (m³): "))

volume_inicial = float(input("Volume inicial disponível (m³): "))

perda = float(input("Percentual de perda na tubulação (%): "))

custo = float(input("Custo de bombeamento por m³ (R$): "))

bairros.append(nome)
demandas.append(demanda)
capacidades.append(capacidade)
volumes_iniciais.append(volume_inicial)
perdas_percentuais.append(perda)
custos_bombeamento.append(custo)

# =====
# PROCESSAMENTO DOS DADOS
# =====

volumes_entregues = []
volumes_perdidos = []
custos_diarios = []
autonomias = []
riscos = []

for i in range(N):

    # Volume perdido
    volume_perdido = demandas[i] * (perdas_percentuais[i] / 100)

    # Volume efetivamente entregue
    volume_entregue = demandas[i] - volume_perdido

    # Custo diário de bombeamento
    custo_diario = demandas[i] * custos_bombeamento[i]

    # Autonomia do reservatório
    autonomia = volumes_iniciais[i] / demandas[i]

    # Classificação do risco
    if autonomia > 3:
        risco = "Baixo"
    elif autonomia >= 1:
        risco = "Médio"
    else:

```

```

        risco = "Alto"

# Armazenamento
volumes_perdidos.append(volume_perdido)
volumes_entregues.append(volume_entregue)
custos_diarios.append(custo_diario)
autonomias.append(autonomia)
riscos.append(risco)

# =====
# CRIAÇÃO DA TABELA
# =====

df = pd.DataFrame({
    "Bairro": bairros,
    "Demanda (m³)": demandas,
    "Capacidade Reservatório (m³)": capacidades,
    "Volume Inicial (m³)": volumes_iniciais,
    "Perda (%)": perdas_percentuais,
    "Volume Perdido (m³)": volumes_perdidos,
    "Volume Entregue (m³)": volumes_entregues,
    "Custo Diário (R$)": custos_diarios,
    "Autonomia (dias)": autonomias,
    "Nível de Risco": riscos
})

# =====
# IDENTIFICAÇÃO DO BAIRRO MAIS CRÍTICO
# =====

indice_critico = autonomias.index(min(autonomias))
bairro_critico = bairros[indice_critico]

# =====
# SAÍDA DOS RESULTADOS
# =====

print("\n")
print("=" * 90)
print("RESULTADOS DA ANÁLISE")
print("=" * 90)

print(df.to_string(index=False))

print("\n" + "=" * 90)
print(f"BAIRRO MAIS CRÍTICO: {bairro_critico}")
print(f"Autonomia: {autonomias[indice_critico]:.2f} dias")
print(f"Nível de risco: {riscos[indice_critico]}")

```

```

print("=" * 90)

# =====
# GRÁFICO 1 - DEMANDA x VOLUME ENTREGUE
# =====

plt.figure(figsize=(12, 6))

x = range(N)
largura = 0.4

plt.bar(
    [i - largura/2 for i in x],
    demandas,
    width=largura,
    label='Demanda',
    color='royalblue'
)

plt.bar(
    [i + largura/2 for i in x],
    volumes_entregues,
    width=largura,
    label='Volume Entregue',
    color='seagreen'
)

plt.xticks(x, bairros)
plt.ylabel("Volume (m³)")
plt.title("Comparação entre Demanda e Volume Entregue")
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()

# =====
# GRÁFICO 2 - CUSTO DIÁRIO POR BAIRRO
# =====

plt.figure(figsize=(12, 6))

plt.bar(
    bairros,
    custos_diarios,
    color='darkorange'
)

plt.ylabel("Custo Diário (R$)")

```

```

plt.title("Custo Diário de Bombeamento por Bairro")
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()

# =====
# GRÁFICO 3 - DISTRIBUIÇÃO DOS NÍVEIS DE RISCO
# =====

contagem_risco = {
    "Baixo": riscos.count("Baixo"),
    "Médio": riscos.count("Médio"),
    "Alto": riscos.count("Alto")
}

plt.figure(figsize=(8, 8))

plt.pie(
    contagem_risco.values(),
    labels=contagem_risco.keys(),
    autopct='%1.1f%%',
    colors=['green', 'gold', 'red'],
    startangle=90
)

plt.title("Distribuição dos Níveis de Risco")

plt.show()

# =====
# RESUMO FINAL
# =====

print("\nRESUMO GERAL DO SISTEMA")
print("-" * 60)

print(f"Total de bairros analisados: {N}")

print(f"Demanda total do sistema: {sum(demandas):.2f} m³")

print(f"Volume total entregue: {sum(volumes_entregues):.2f} m³")

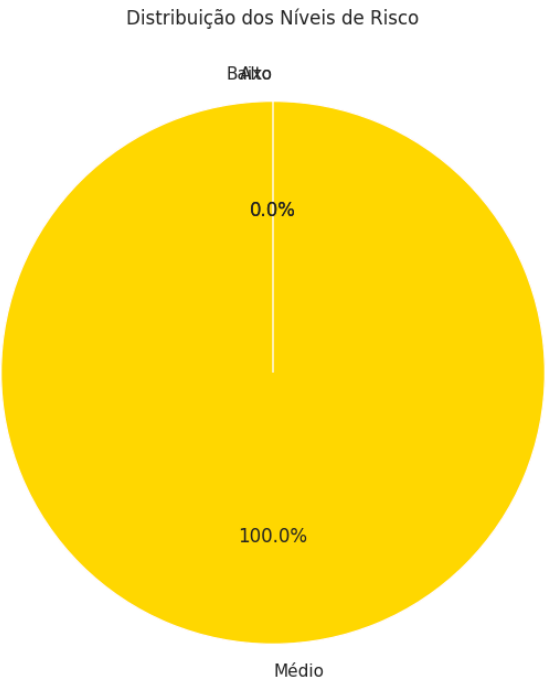
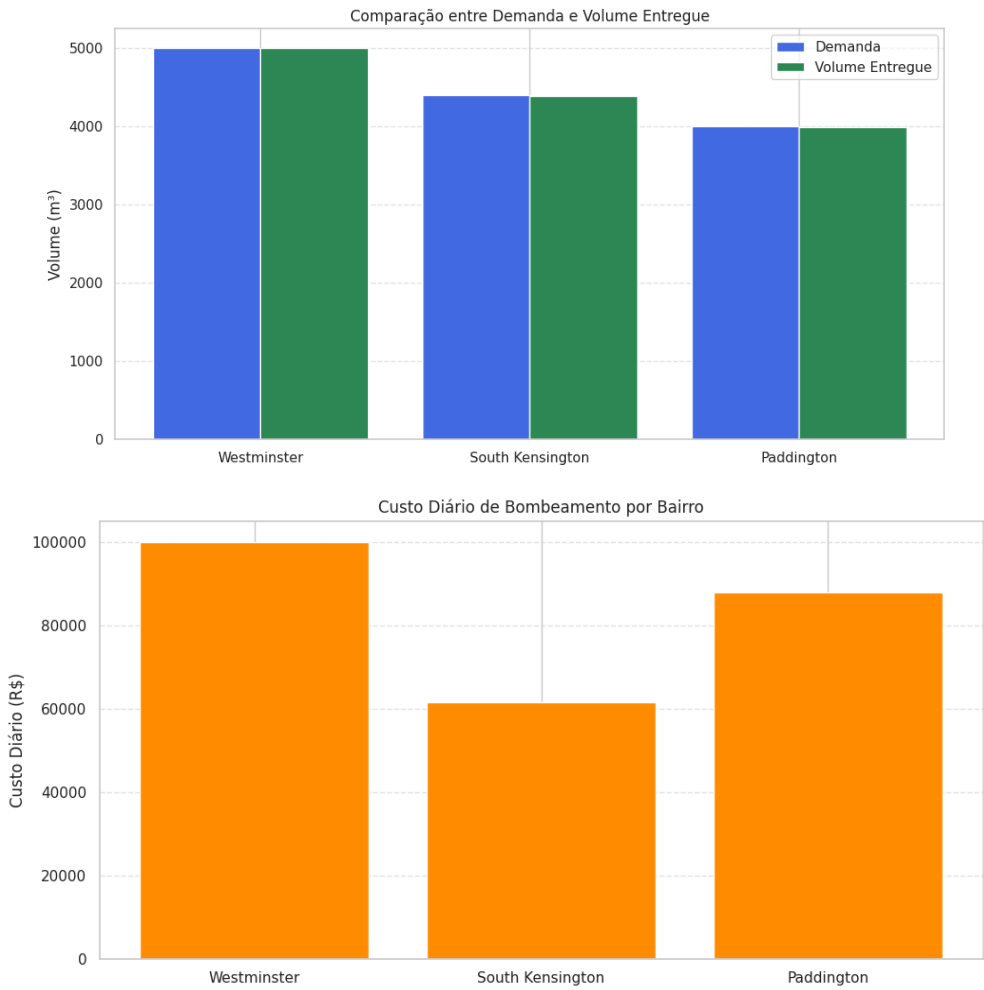
print(f"Perda total de água: {sum(volumes_perdidos):.2f} m³")

print(f"Custo operacional total: R$ {sum(custos_diarios):.2f}")

print("-" * 60)

```

Anexos Chat GPT



LLM 2 | Gemini

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Configuração visual dos gráficos
sns.set_theme(style="whitegrid")

def analisar_abastecimento():
    print("--- Análise de Estabilidade e Custo: Sistema de Abastecimento ---")

    try:
        n = int(input("Informe o número de bairros a serem analisados: "))
    except ValueError:
        print("Erro: Por favor, insira um número inteiro válido.")
        return

    dados_bairros = []

    for i in range(n):
        print(f"\n--- Dados do Bairro {i+1} ---")
        nome = input("Nome do bairro: ")
        demanda = float(input(f"Demanda diária de {nome} (m³): "))
        capacidade = float(input(f"Capacidade do reservatório de {nome} (m³): "))
        vol_inicial = float(input(f"Volume inicial disponível em {nome} (m³): "))
        perda_perc = float(input(f"Percentual de perda na tubulação (0-100%): ")) / 100
        custo_m3 = float(input(f"Custo de bombeamento por m³ (R$): "))

        # Cálculos de Processamento
        vol_perdido = demanda * perda_perc
        vol_entregue = demanda - vol_perdido
        custo_diario = demanda * custo_m3

        # Autonomia baseada no volume disponível atual
        autonomia = vol_inicial / demanda if demanda > 0 else float('inf')

        # Classificação de Risco
        if autonomia > 3:
            risco = "Baixo"
        elif 1 <= autonomia <= 3:
            risco = "Médio"
        else:
            risco = "Alto"
```

```

dados_bairros.append({
    "Bairro": nome,
    "Demanda (m³)": demanda,
    "Volume Entregue (m³)": vol_entregue,
    "Volume Perdido (m³)": vol_perdido,
    "Custo Diário (R$)": custo_diario,
    "Autonomia (Dias)": round(autonomia, 2),
    "Nível de Risco": risco
})

# Criação do DataFrame
df = pd.DataFrame(dados_bairros)

# --- Saídas Obrigatórias ---

# 1. Tabela de Resultados
print("\n" + "="*30)
print("TABELA DE RESULTADOS POR BAIRRO")
print("="*30)
display(df)

# 2. Identificação do Bairro Mais Crítico
# Critério: Menor autonomia
bairro_critico = df.loc[df['Autonomia (Dias)'].idxmin()]
print(f"\n⚠ IDENTIFICAÇÃO CRÍTICA:")
print(f"O bairro mais crítico é '{bairro_critico['Bairro']}' com autonomia de apenas {bairro_critico['Autonomia (Dias)'} dias.")

# 3. Gráficos
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
plt.subplots_adjust(hspace=0.4)

# Gráfico A: Demanda vs Volume Entregue
df_melted = df.melt(id_vars='Bairro', value_vars=['Demanda (m³)', 'Volume Entregue (m³)'], var_name='Tipo', value_name='Volume')
sns.barplot(data=df_melted, x='Bairro', y='Volume', hue='Tipo', ax=axes[0, 0])
axes[0, 0].set_title("Comparativo: Demanda vs Volume Efetivamente Entregue")

# Gráfico B: Custo Diário por Bairro
sns.barplot(data=df, x='Bairro', y='Custo Diário (R$)', palette='viridis', ax=axes[0, 1])
axes[0, 1].set_title("Custo Diário de Bombeamento por Bairro")

# Gráfico C: Distribuição dos Níveis de Risco
risco_counts = df['Nível de Risco'].value_counts()
axes[1, 0].pie(risco_counts, labels=risco_counts.index, autopct='%1.1f%%',
colors=['#66b3ff', '#ffcc99', '#ff9999'])
axes[1, 0].set_title("Distribuição dos Níveis de Risco no Sistema")

```



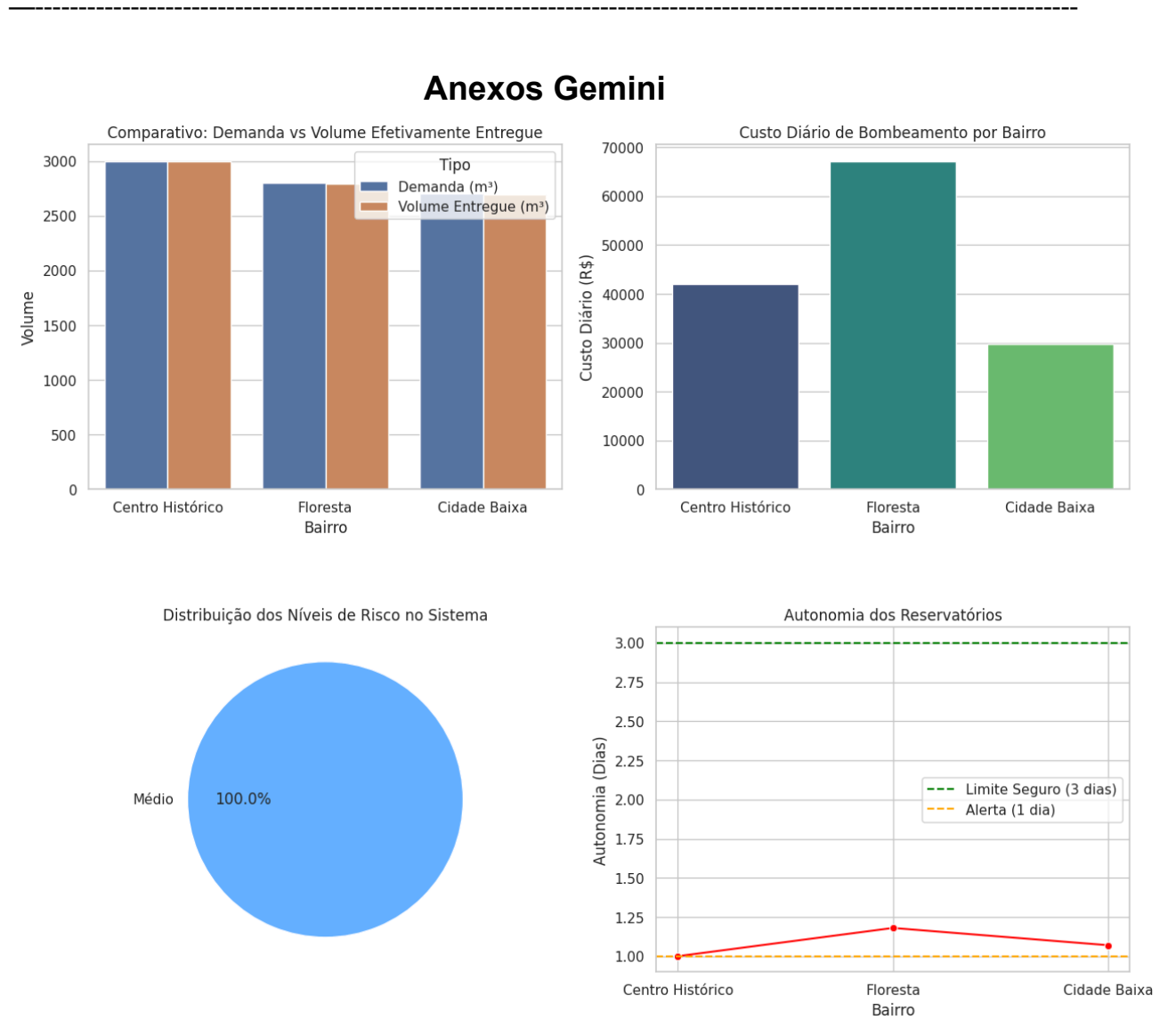
```

# Gráfico D: Autonomia (Visualização Extra)
sns.lineplot(data=df, x='Bairro', y='Autonomia (Dias)', marker='o', color='red', ax=axes[1,
1])
axes[1, 1].axhline(3, ls='--', color='green', label='Limite Seguro (3 dias)')
axes[1, 1].axhline(1, ls='--', color='orange', label='Alerta (1 dia)')
axes[1, 1].set_title("Autonomia dos Reservatórios")
axes[1, 1].legend()

plt.show()

# Executar o programa
analisar_abastecimento()

```



LLM 3 | Claude

```
# =====  
# ANÁLISE DE ESTABILIDADE E CUSTO EM SISTEMAS DE ABASTECIMENTO  
# =====
```

```
import matplotlib.pyplot as plt  
import matplotlib.patches as mpatches  
import numpy as np  
from IPython.display import display  
import pandas as pd
```

```
# ----- CORES E ESTILO -----
```

```
plt.rcParams.update({  
    'figure.facecolor': '#0d1117',  
    'axes.facecolor': '#161b22',  
    'axes.edgecolor': '#30363d',  
    'axes.labelcolor': '#e6edf3',  
    'xtick.color': '#8b949e',  
    'ytick.color': '#8b949e',  
    'text.color': '#e6edf3',  
    'grid.color': '#21262d',  
    'grid.linestyle': '--',  
    'grid.alpha': 0.6,  
    'font.family': 'monospace',  
})
```

```
COR_BAIXO = '#3fb950' # verde  
COR_MEDIO = '#d29922' # amarelo  
COR_ALTO = '#f85149' # vermelho  
COR_DEMANDA = '#58a6ff'  
COR_ENTREGUE = '#3fb950'  
COR_CUSTO = '#bc8cff'
```

```
def cor_risco(nivel):  
    return {'Baixo': COR_BAIXO, 'Médio': COR_MEDIO, 'Alto': COR_ALTO}[nivel]
```

```
# ----- BANNER -----
```

```
def banner():  
    print("\n" + "="*60)  
    print(" 💧 ANÁLISE DE ABASTECIMENTO DE ÁGUA")  
    print(" Sistema de Monitoramento Municipal")  
    print("="*60 + "\n")
```

```
# ----- COLETA DE DADOS -----
```

```
def coletar_dados():  
    banner()
```

```

while True:
    try:
        n = int(input("Número de bairros a analisar: "))
        if n < 1:
            print(" ⚠ Informe ao menos 1 bairro.")
        else:
            break
    except ValueError:
        print(" ⚠ Digite um número inteiro válido.")

bairros = []
for i in range(1, n + 1):
    print(f"\n{'-'*40}")
    print(f" BAIRRO {i} de {n}")
    print(f"{'-'*40}")
    nome = input(f" Nome do bairro {i}: ").strip() or f"Bairro {i}"

    def ler_float(prompt, minv=0):
        while True:
            try:
                v = float(input(f" {prompt}"))
                if v < minv:
                    print(f" ⚠ Valor deve ser ≥ {minv}.")
            except ValueError:
                print(" ⚠ Digite um número válido.")
            else:
                return v

    demanda = ler_float("Demanda diária estimada (m³): ", 0.01)
    capacidade = ler_float("Capacidade do reservatório (m³): ", 0.01)
    volume_ini = ler_float("Volume inicial disponível (m³): ", 0)
    while True:
        perda = ler_float("Percentual de perda na tubulação (%): ", 0)
        if perda >= 100:
            print(" ⚠ Perda deve ser menor que 100%.")
        else:
            break
    custo_bom = ler_float("Custo de bombeamento por m³ (R$): ", 0)

    bairros.append({
        'nome': nome,
        'demanda': demanda,
        'capacidade': capacidade,
        'volume_ini': volume_ini,
        'perda_pct': perda,
        'custo_bom': custo_bom,
    })

```

```

return bairros

# ----- PROCESSAMENTO -----
def processar(bairros):
    resultados = []
    for b in bairros:
        fator_perda = b['perda_pct'] / 100
        vol_perdido = b['demanda'] * fator_perda
        vol_entregue = b['demanda'] * (1 - fator_perda)
        custo_diario = b['demanda'] * b['custo_bom']

        # autonomia baseada no volume inicial vs demanda líquida
        if vol_entregue > 0:
            autonomia = b['volume_ini'] / vol_entregue
        else:
            autonomia = float('inf')

        if autonomia > 3:
            risco = 'Baixo'
        elif autonomia >= 1:
            risco = 'Médio'
        else:
            risco = 'Alto'

        resultados.append({
            'nome': b['nome'],
            'demanda': b['demanda'],
            'capacidade': b['capacidade'],
            'volume_ini': b['volume_ini'],
            'perda_pct': b['perda_pct'],
            'vol_perdido': vol_perdido,
            'vol_entregue': vol_entregue,
            'custo_diario': custo_diario,
            'autonomia': autonomia,
            'risco': risco,
        })
    return resultados

# ----- TABELA DE RESULTADOS -----
def imprimir_tabela(resultados):
    print("\n" + "="*60)
    print(" 📊 RESULTADOS POR BAIRRO")
    print("="*60)
    df = pd.DataFrame([
        'Bairro': r['nome'],
        'Demanda (m³)': f"{r['demanda']:.1f}",
        'Entregue (m³)': f"{r['vol_entregue']:.1f}",
        'Perdido (m³)': f"{r['vol_perdido']:.1f}",
    ])

```

```

'Custo Diário': f'R$ {r["custo_diario"]:.2f}',
'Autonomia (d)': f"{r['autonomia']:.2f}" if r['autonomia'] != float('inf') else '∞',
'Risco': r['risco'],
} for r in resultados])
display(df.to_string(index=False))

```

```

# Bairro mais crítico = maior risco → menor autonomia
critico = min(resultados, key=lambda r: r['autonomia'])
print(f"\n 🚨 BAIRRO MAIS CRÍTICO: {critico['nome'].upper()}")
print(f"    Risco: {critico['risco']} | "
      f"Autonomia: {critico['autonomia']:.2f} dia(s) | "
      f"Custo: R$ {critico['custo_diario']:.2f}")
print("="*60)

```

----- GRÁFICOS -----

```
def gerar_graficos(resultados):
```

```

    nomes    = [r['nome'] for r in resultados]
    demandas = [r['demanda'] for r in resultados]
    entregues = [r['vol_entregue'] for r in resultados]
    custos   = [r['custo_diario'] for r in resultados]
    riscos    = [r['risco'] for r in resultados]
    cores_bar = [cor_risco(r) for r in riscos]

```

```

x = np.arange(len(nomes))
w = 0.38

```

```

fig, axes = plt.subplots(2, 2, figsize=(16, 11))
fig.suptitle('Sistema de Abastecimento de Água — Painel de Controle',
             fontsize=15, fontweight='bold', color='#e6edf3', y=0.98)
fig.patch.set_facecolor('#0d1117')

```

— Gráfico 1: Demanda vs Entrega —————

```

ax1 = axes[0, 0]
b1 = ax1.bar(x - w/2, demandas, w, label='Demanda', color=COR_DEMANDA,
alpha=0.9, zorder=3)
b2 = ax1.bar(x + w/2, entregues, w, label='Entrega', color=COR_ENTREGA, alpha=0.9,
zorder=3)
ax1.set_title('Demanda × Volume Entrega (m³)', fontweight='bold', pad=10)
ax1.set_xticks(x); ax1.set_xticklabels(nomes, rotation=30, ha='right', fontsize=9)
ax1.set_ylabel('Volume (m³)'); ax1.legend(); ax1.grid(axis='y', zorder=0)
for bar in list(b1) + list(b2):
    h = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2, h + 0.5, f'{h:.0f}',
             ha='center', va='bottom', fontsize=7.5, color='#8b949e')

```

— Gráfico 2: Custo Diário —————

```
ax2 = axes[0, 1]
```

```
bars = ax2.bar(x, custos, color=cores_bar, alpha=0.88, zorder=3, edgecolor='#0d1117',
linewidth=0.8)
```

```
ax2.set_title('Custo Diário de Bombeamento (R$)', fontweight='bold', pad=10)
ax2.set_xticks(x); ax2.set_xticklabels(nomes, rotation=30, ha='right', fontsize=9)
```

```
ax2.set_ylabel('R$'); ax2.grid(axis='y', zorder=0)
```

```
for bar in bars:
```

```
    h = bar.get_height()
```

```
    ax2.text(bar.get_x() + bar.get_width()/2, h + max(custos)*0.01,
            f'R$ {h:.2f}', ha='center', va='bottom', fontsize=7.5, color='#8b949e')
```

```
# — Gráfico 3: Distribuição de Riscos (pizza) —————
```

```
ax3 = axes[1, 0]
```

```
contagem = {'Baixo': riscos.count('Baixo'),
            'Médio': riscos.count('Médio'),
            'Alto': riscos.count('Alto')}
```

```
labels_ativos = [k for k, v in contagem.items() if v > 0]
```

```
vals_ativos = [contagem[k] for k in labels_ativos]
```

```
cores_pizza = [cor_risco(k) for k in labels_ativos]
```

```
explode = [0.05] * len(labels_ativos)
```

```
wedges, texts, autotexts = ax3.pie(
    vals_ativos, labels=labels_ativos, colors=cores_pizza,
    autopct='%1.0f%%', startangle=140, explode=explode,
    wedgeprops={'edgecolor': '#0d1117', 'linewidth': 1.5},
    textprops={'color': '#e6edf3', 'fontsize': 10})
```

```
for at in autotexts:
```

```
    at.set_fontsize(9); at.set_color('#0d1117'); at.set_fontweight('bold')
```

```
ax3.set_title('Distribuição dos Níveis de Risco', fontweight='bold', pad=10)
```

```
# — Gráfico 4: Autonomia por Bairro —————
```

```
ax4 = axes[1, 1]
```

```
autonomias = [min(r['autonomia'], 10) for r in resultados] # cap p/ leitura
```

```
bars4 = ax4.bar(x, autonomias, color=cores_bar, alpha=0.88, zorder=3,
edgecolor='#0d1117', linewidth=0.8)
```

```
ax4.axhline(3, color=COR_BAIXO, linestyle='--', linewidth=1.2, label='Limite Baixo (3d)',
zorder=4)
```

```
ax4.axhline(1, color=COR_ALTO, linestyle='--', linewidth=1.2, label='Limite Alto (1d)',
zorder=4)
```

```
ax4.set_title('Autonomia do Reservatório (dias)', fontweight='bold', pad=10)
```

```
ax4.set_xticks(x); ax4.set_xticklabels(nomes, rotation=30, ha='right', fontsize=9)
```

```
ax4.set_ylabel('Dias'); ax4.legend(fontsize=8); ax4.grid(axis='y', zorder=0)
```

```
for i, (bar, r) in enumerate(zip(bars4, resultados)):
```

```
    h = bar.get_height()
```

```
    label = '∞' if r['autonomia'] == float('inf') else f'{r["autonomia"]:.1f}d'
```

```
    ax4.text(bar.get_x() + bar.get_width()/2, h + 0.05,
            label, ha='center', va='bottom', fontsize=7.5, color='#8b949e')
```

```
# Legenda de risco compartilhada (rodapé)
```

```

patches = [mpatches.Patch(color=COR_BAIXO, label='Risco Baixo (>3d)'),
            mpatches.Patch(color=COR_MEDIO, label='Risco Médio (1–3d)'),
            mpatches.Patch(color=COR_ALTO, label='Risco Alto (<1d)')]
fig.legend(handles=patches, loc='lower center', ncol=3,
           framealpha=0.15, fontsize=9, bbox_to_anchor=(0.5, 0.01))

plt.tight_layout(rect=[0, 0.05, 1, 0.97])
plt.savefig('abastecimento_painel.png', dpi=150, bbox_inches='tight',
           facecolor='#0d1117')
plt.show()
print("\n 📁 Painel salvo como 'abastecimento_painel.png'")

# ----- MAIN -----
def main():
    bairros = coletar_dados()
    resultados = processar(bairros)
    imprimir_tabela(resultados)
    gerar_graficos(resultados)

main()

```

Anexos Claude



Opinião e comparação final das 3 LLM's

Da minha experiência de programação: O Chat GPT foi o único que escreveu de forma “imperativa”, sem separar em funções.. Tanto o Gemini como Claude ambos modularizaram em funções (Claude chamou o main() e o Gemini chamou a função que ele criou). O Gemini gerou um código um pouco mais enxuto, mas na minha percepção, um pouco mais difícil de ler. O Claude tratou melhor erros também, tem vários “except” no código. Por isso o Claude me parece o mais seguro (tanto é que é o mais recomendado para geração de código hoje em dia). Por esses motivos, creio que o melhor resultado tenha sido o código produzido pelo Claude. Todos geraram bons resultados, mas o da Claude foi o mais completo apesar das 3 LLM's terem o mesmo contexto e prompt.